

BIP-54: Consensus Cleanup

Random bullshit go

Agenda

- Content and history

Agenda

- Content and history
- Timewarp

Agenda

- Content and history
- Timewarp
- Legacy sigops

Agenda

- Content and history
- Timewarp
- Legacy sigops
- Merkle trees

Agenda

- Content and history
- Timewarp
- Legacy sigops
- Merkle trees
- Coinbase uniqueness

Agenda

- Content and history
- Timewarp
- Legacy sigops
- Merkle trees
- Coinbase uniqueness
- Mining impact

Context

Four main changes

Timewarp

difficulty can be forced
down

Legacy sigops

pathological validation
cost

64-byte txs

Merkle leaf/node
ambiguity

Duplicate txids

future BIP30 problem

A lot of unrelated stuff in one bip whew

History and status

1. **2019** - Matt Corallo proposes the Great Consensus Cleanup
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.

History and status

1. **2019** - Matt Corallo proposes the Great Consensus Cleanup
2. **2023-24** - Antoine Poinot reworks the proposal
- 3.
- 4.
- 5.
- 6.
- 7.

History and status

1. **2019** - Matt Corallo proposes the Great Consensus Cleanup
2. **2023-24** - Antoine Poinot reworks the proposal
3. **26 March 2025** - consolidated draft published
- 4.
- 5.
- 6.
- 7.

History and status

1. **2019** - Matt Corallo proposes the Great Consensus Cleanup
2. **2023-24** - Antoine Poinot reworks the proposal
3. **26 March 2025** - consolidated draft published
4. **11 April 2025** - assigned BIP 54
- 5.
- 6.
- 7.

History and status

1. **2019** - Matt Corallo proposes the Great Consensus Cleanup
2. **2023-24** - Antoine Poinot reworks the proposal
3. **26 March 2025** - consolidated draft published
4. **11 April 2025** - assigned BIP 54
5. **February 2026** - Bitcoin Inquisition activates it on signet
- 6.
- 7.

History and status

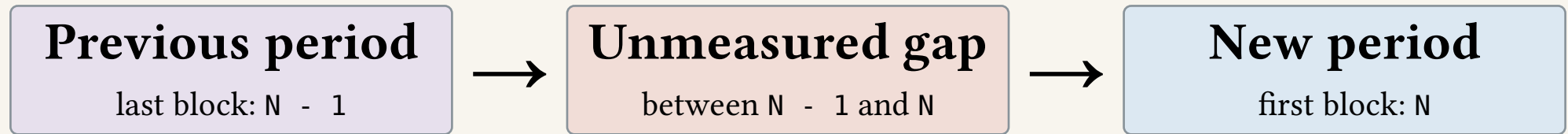
1. **2019** - Matt Corallo proposes the Great Consensus Cleanup
2. **2023-24** - Antoine Poinot reworks the proposal
3. **26 March 2025** - consolidated draft published
4. **11 April 2025** - assigned BIP 54
5. **February 2026** - Bitcoin Inquisition activates it on signet
6. **22 May 2026** - version 1.0.0, status Complete
- 7.

History and status

1. **2019** - Matt Corallo proposes the Great Consensus Cleanup
2. **2023-24** - Antoine Poinot reworks the proposal
3. **26 March 2025** - consolidated draft published
4. **11 April 2025** - assigned BIP 54
5. **February 2026** - Bitcoin Inquisition activates it on signet
6. **22 May 2026** - version 1.0.0, status Complete
7. **Today** - Bitcoin Core PR open for regtest; mainnet activation unselected

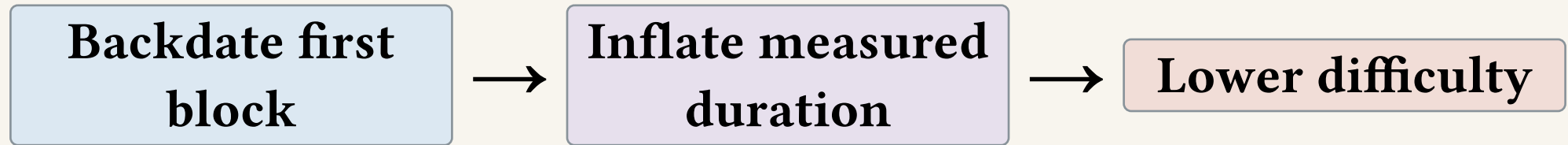
Difficulty adjustment

Retarget gap



Difficulty retargets every 2016 blocks from the period's first and last timestamps.
The boundary interval drops out of the calculation.

Timewarp



A majority-hashrate attacker repeats this at each retarget.
Difficulty reaches the minimum in roughly 38 days; blocks then accelerate without bound.

Timestamp constraints

First block of period

$$T_N \geq T_{N-1} - 7200$$

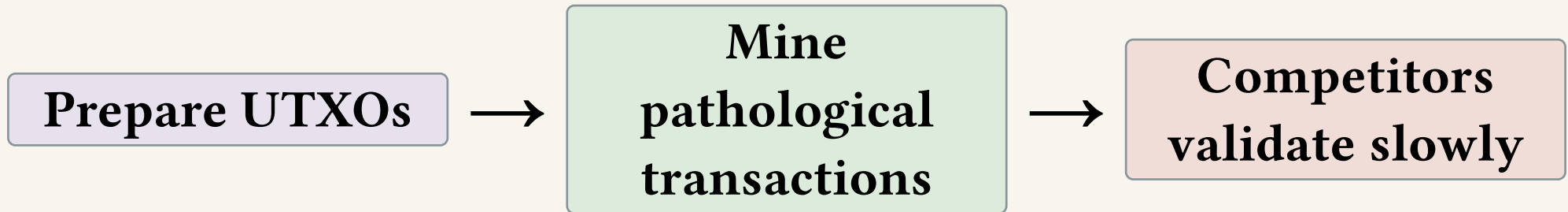
Last block of period

$$T_N \geq T_{N-2015}$$

The first rule caps the timewarp at two hours. The second prevents a negative period (Murch-Zawy).

Validation time

Legacy sigops



A sigop is a CHECKSIG or CHECKMULTISIG operation in Bitcoin Script.
Legacy Script can concentrate expensive checks inside a few transactions.

Validation time

Validation cost

- About 10 minutes on a Core i5-12500H in a 2026 test

Validation cost

- About 10 minutes on a Core i5-12500H in a 2026 test
- About 29 minutes in a comparable test on an older Xeon

Validation cost

- About 10 minutes on a Core i5-12500H in a 2026 test
- About 29 minutes in a comparable test on an older Xeon
- The attacker prepares suitable UTXOs in earlier blocks

Validation cost

- About 10 minutes on a Core i5-12500H in a 2026 test
- About 29 minutes in a comparable test on an older Xeon
- The attacker prepares suitable UTXOs in earlier blocks
- A miner starts the next block while competitors validate the slow block

Validation cost

- About 10 minutes on a Core i5-12500H in a 2026 test
- About 29 minutes in a comparable test on an older Xeon
- The attacker prepares suitable UTXOs in earlier blocks
- A miner starts the next block while competitors validate the slow block
- Under some conditions, the head start can cover the preparation cost

2500 per transaction

Opcode form	BIP 54 sigops
CHECKSIG / CHECKSIGVERIFY	1
OP_1...OP_16 then CHECKMULTISIG	1...16
Other CHECKMULTISIG forms	20
Transaction total	0...2500 valid; >2500 invalid

Count each input's scriptSig, spent scriptPubKey, and P2SH redeemScript.

This BIP16-style count includes unreachable opcodes.

The limit cuts worst-case validation time by roughly 40 times.

Merkle trees

The 64-byte ambiguity

Transaction leaf

`txid = SHA256d(serialized_tx)`

Inner node

`SHA256d(left_hash || right_hash)`

An inner-node preimage is $32 + 32 = 64$ bytes.

A 64-byte witness-stripped transaction can occupy either interpretation.

Merkle proof rule

- Ambiguous leaves can forge an apparent SPV inclusion with less work than a SHA256 collision

Merkle proof rule

- Ambiguous leaves can forge an apparent SPV inclusion with less work than a SHA256 collision
- BIP 54 makes transactions with exactly 64 witness-stripped bytes invalid

Merkle proof rule

- Ambiguous leaves can forge an apparent SPV inclusion with less work than a SHA256 collision
- BIP 54 makes transactions with exactly 64 witness-stripped bytes invalid
- Current Bitcoin Core policy already excludes them from relay and block templates

Merkle proof rule

- Ambiguous leaves can forge an apparent SPV inclusion with less work than a SHA256 collision
- BIP 54 makes transactions with exactly 64 witness-stripped bytes invalid
- Current Bitcoin Core policy already excludes them from relay and block templates
- The last one appeared on-chain in 2016

Merkle proof rule

- Ambiguous leaves can forge an apparent SPV inclusion with less work than a SHA256 collision
- BIP 54 makes transactions with exactly 64 witness-stripped bytes invalid
- Current Bitcoin Core policy already excludes them from relay and block templates
- The last one appeared on-chain in 2016
- Present-day 64-byte transactions create anyone-can-spend or provably unspendable outputs

Coinbase uniqueness

BIP30 and BIP34

- Coinbase transactions create subsidy and fees and carry a txid

BIP30 and BIP34

- Coinbase transactions create subsidy and fees and carry a txid
- **BIP30** protects unspent outputs from a duplicate txid

BIP30 and BIP34

- Coinbase transactions create subsidy and fees and carry a txid
- **BIP30** protects unspent outputs from a duplicate txid
- Two historical blocks remain explicit exceptions

BIP30 and BIP34

- Coinbase transactions create subsidy and fees and carry a txid
- **BIP30** protects unspent outputs from a duplicate txid
- Two historical blocks remain explicit exceptions
- **BIP34** encodes block height in the coinbase scriptSig

BIP30 and BIP34

- Coinbase transactions create subsidy and fees and carry a txid
- **BIP30** protects unspent outputs from a duplicate txid
- Two historical blocks remain explicit exceptions
- **BIP34** encodes block height in the coinbase scriptSig
- Bitcoin Core uses BIP34 to skip most explicit BIP30 checks

Future duplicate height

- Some pre-BIP34 coinbases begin with bytes that encode future heights

Future duplicate height

- Some pre-BIP34 coinbases begin with bytes that encode future heights
- The earliest duplicate candidate occurs at height **1,983,702**

Future duplicate height

- Some pre-BIP34 coinbases begin with bytes that encode future heights
- The earliest duplicate candidate occurs at height **1,983,702**
- Explicit BIP30 validation would resume before that height

Future duplicate height

- Some pre-BIP34 coinbases begin with bytes that encode future heights
- The earliest duplicate candidate occurs at height **1,983,702**
- Explicit BIP30 validation would resume before that height
- BIP30 needs the UTXO set and complicates designs such as Utreexo

Future duplicate height

- Some pre-BIP34 coinbases begin with bytes that encode future heights
- The earliest duplicate candidate occurs at height **1,983,702**
- Explicit BIP30 validation would resume before that height
- BIP30 needs the UTXO set and complicates designs such as Utreexo
- A height-bound coinbase provides permanent uniqueness

Height-bound coinbase

nLockTime

$N - 1$

input nSequence

must not equal 0xffffffff

nSequence activates the locktime.

nLockTime = $N - 1$ becomes final at height N , binding the transaction to that block height.

Operational impact

Nodes and applications

- Upgraded nodes enforce all four rules after activation

Nodes and applications

- Upgraded nodes enforce all four rules after activation
- Standard wallet transactions fit the new limits

Nodes and applications

- Upgraded nodes enforce all four rules after activation
- Standard wallet transactions fit the new limits
- Script tooling enforces the 2500 legacy-sigop ceiling

Nodes and applications

- Upgraded nodes enforce all four rules after activation
- Standard wallet transactions fit the new limits
- Script tooling enforces the 2500 legacy-sigop ceiling
- Transaction builders pad a 64-byte stripped transaction

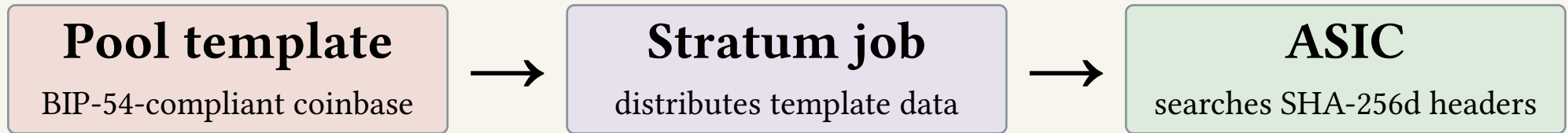
Nodes and applications

- Upgraded nodes enforce all four rules after activation
- Standard wallet transactions fit the new limits
- Script tooling enforces the 2500 legacy-sigop ceiling
- Transaction builders pad a 64-byte stripped transaction
- SPV, bridge, and sidechain code gets unambiguous Merkle proofs

Nodes and applications

- Upgraded nodes enforce all four rules after activation
- Standard wallet transactions fit the new limits
- Script tooling enforces the 2500 legacy-sigop ceiling
- Transaction builders pad a 64-byte stripped transaction
- SPV, bridge, and sidechain code gets unambiguous Merkle proofs
- Utreexo implementations can skip future BIP30 validation

Mining pools



The pool sets `nLockTime = block height - 1` and `nSequence != 0xffffffff`.

It uses `curtime` or `mintime` from `getblocktemplate`.

Pools continue rolling `extranonce` in `scriptSig`.

Deployment readiness

- Bitcoin Core 29.0+ templates use compatible timestamps

Deployment readiness

- Bitcoin Core 29.0+ templates use compatible timestamps
- Bitcoin Core 30.0+ policy enforces the 2500-sigop limit

Deployment readiness

- Bitcoin Core 29.0+ templates use compatible timestamps
- Bitcoin Core 30.0+ policy enforces the 2500-sigop limit
- Bitcoin Core 0.16.1+ policy excludes 64-byte stripped transactions

Deployment readiness

- Bitcoin Core 29.0+ templates use compatible timestamps
- Bitcoin Core 30.0+ policy enforces the 2500-sigop limit
- Bitcoin Core 0.16.1+ policy excludes 64-byte stripped transactions
- Pool software adds the coinbase locktime and sequence fields

Deployment readiness

- Bitcoin Core 29.0+ templates use compatible timestamps
- Bitcoin Core 30.0+ policy enforces the 2500-sigop limit
- Bitcoin Core 0.16.1+ policy excludes 64-byte stripped transactions
- Pool software adds the coinbase locktime and sequence fields
- Mainnet activation mechanism and parameters remain unselected

Takeaways

- BIP 54 packages four consensus fixes

Takeaways

- BIP 54 packages four consensus fixes
- Boundary timestamps close timewarp

Takeaways

- BIP 54 packages four consensus fixes
- Boundary timestamps close timewarp
- Per-transaction sigops bound validation cost

Takeaways

- BIP 54 packages four consensus fixes
- Boundary timestamps close timewarp
- Per-transaction sigops bound validation cost
- A 64-byte ban removes the Merkle ambiguity

Takeaways

- BIP 54 packages four consensus fixes
- Boundary timestamps close timewarp
- Per-transaction sigops bound validation cost
- A 64-byte ban removes the Merkle ambiguity
- Coinbase locktime guarantees unique future txids

Takeaways

- BIP 54 packages four consensus fixes
- Boundary timestamps close timewarp
- Per-transaction sigops bound validation cost
- A 64-byte ban removes the Merkle ambiguity
- Coinbase locktime guarantees unique future txids
- Pools implement the only mining-specific change

Operational impact

Q&A